

Week06 | 课时 3 | Partition / Backfill / Replay: 把补数做成可控动作

本课导航

补数不是“再跑一遍”，而是对明确边界的受控重算	1
这节课解决什么问题	1
参考学习时间	1
学完这一讲，你应该能做到什么	2
本课产出	2
先看一张时间线	2
1. partition 是系统承认的恢复边界	2
2. 为什么先用 daily partition	2
3. 五个恢复动作不要混用	3
4. Backfill plan 最少包含什么	4
5. 安全制造缺口	5
6. 补数后的验收顺序不能乱	5
7. 企业里 backfill 最容易出事故的 6 个点	6
自检清单	6
课后最小行动	6
延伸阅读	6

补数不是“再跑一遍”，而是对明确边界的受控重算

这一讲把 Week03 的 replay/backfill 思维升级成 Week06 的 asset-level partition recovery。

这节课解决什么问题

因为某天的数据缺了一部分，工程同学直接全量重跑。结果 raw 表多了一批重复 ticket，Week05 的 P1 增长 KPI 比昨天高了 15%，Week08 的检索索引又把重复内容嵌入进去。后来排查发现，问题不在于他不会跑脚本，而在于他没有明确补数边界：补哪个 partition、输入 manifest 是哪版、当前输出有多少、补完以后谁来验收、下游什么时候可以解除阻断。

这类事故的本质不是“脚本坏了”，而是恢复动作没有被工程化。**补数不是勇敢重跑，而是受控恢复。**

如果没有 partition boundary，backfill 很容易变成“全量重跑一次试试”。这在 AI 数据链路里很危险：

- 可能重复写入 raw / silver；
- 可能覆盖已经发布的 baseline；
- 可能让下游指标和索引版本漂移；
- 可能无法解释补数影响范围；
- 可能让团队只能口头说“应该修好了”，却没有证据交接。

参考学习时间

50—60 分钟

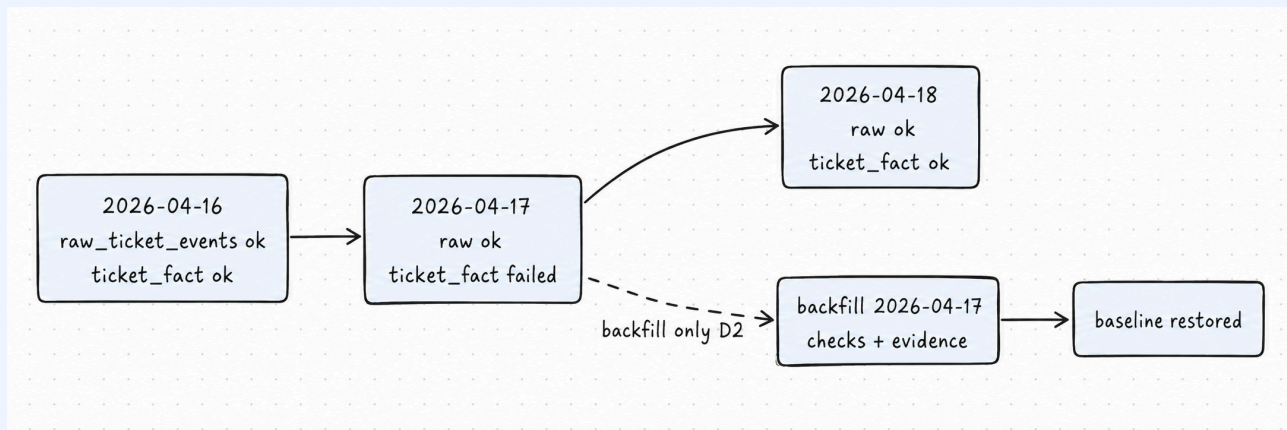
学完这一讲，你应该能做到什么

1. 说明 daily partition 为什么是 Student Core 默认。
2. 区分 retry、rerun、replay、restore、backfill。
3. 写出一个 partition-level backfill dry-run plan。
4. 解释 backfill 后为什么必须跑 checks 和 evidence。
5. 设计安全缺口模拟方式，而不是危险删除主表。

本课产出

- docs/blueprints/week06/week06-partition-backfill-strategy.md
- reports/week06/backfill/

先看一张时间线



这张图想让你看懂一件事：补数不是把整条链路倒回去重跑，而是找到失败的时间窗口，只对这个窗口做受控重算。Week06 的重点不是追求更复杂的调度，而是让恢复动作有边界、有输入证据、有输出验收。

1. partition 是系统承认的恢复边界

对小白来说，partition 可以理解成系统正式承认的“时间窗口抽屉”。每天一个抽屉，抽屉里应该有哪些输入、产出、检查和证据，都可以被独立确认。

它不是为了炫技，也不是为了分目录好看，而是为了回答四个工程问题：

- 这个时间窗口的数据是不是完整？
- 失败后只补这个窗口行不行？
- 下游是否可以只阻断这个窗口？
- 运行证据能否指向具体窗口？

💡 判断是否需要 partition 的简单标准

如果一次失败会影响多个日期、多个下游和多份证据，但你只能说“我再跑一遍看看”，那就说明系统还没有把恢复边界建出来。

2. 为什么先用 daily partition

Student Core 默认 daily partition，是因为它最容易解释、最容易回放、最容易和业务时间对齐。客服、运营、BI 报表通常也按天复盘，课程里的 2026-04-17 就是一个固定课堂分区。

分区方式	适合场景	Week06 判断
daily	大多数批处理、指标和报告	默认选项，和客服运营日、报表日、ingest 日期对齐
batch id	一次 manifest / batch 边界非常清晰	写进 metadata, 必要时辅助排查
manifest id	需要严格重放某次输入声明	适合 replay report, 不替代 daily partition
product line	多业务线隔离	Instructor Scale 或后续扩展
dynamic / multi partitions	大规模生产编排	先讲边界，不作为本周必做
hourly	近实时、高频增量	容易把初学者带进复杂调度，Student Core 不默认

daily partition 的好处是：足够小，可以局部补数；足够大，不会把初学者拖入 hourly、dynamic partition、多维分区的复杂性。课程先把“边界意识”做扎实，再谈更细粒度的生产编排。

在当前 `omnisupport-copilot` Week6 项目实现里，课堂默认分区是 `2026-04-17`，环境变量落点是：

```
# 可执行：已与项目 runbook 对齐
WEEK06_PARTITION_START_DATE=2026-03-01
WEEK06_DEFAULT_PARTITION=2026-04-17
WEEK06_REPORT_DIR=reports/week06
```

怎么看输出：如果你的 `dry-run plan` 不是围绕 `2026-04-17` 生成，先检查 `.env.local` 是否从 `infra/env/.env.example` 同步过。

3. 五个恢复动作不要混用

动作	一句话	什么时候用	不能做什么
retry	同一次失败再试一次	网络抖动、临时连接、对象存储瞬时失败	不能改变输入、代码或分区
rerun	重新跑同一个 job	资源配置或代码修好后，需要复跑同一作业定义	不能绕过幂等，不能假装影响范围没变
replay	重放同一批输入	要复现某次 ingest 或验证 manifest 声明	不能换 manifest，不能偷偷补新数据
restore	回到已知好状态	当前状态不可信，需要先回到可用 baseline	不能假装根因已经修复
backfill	补历史空洞	某个 partition 缺失、延迟或需要重算	不能无边界全量重跑

⚠ 没有 partition boundary, 补数只能靠经验

如果你说不清 window_start、window_end、upstream_manifest_id 和 idempotency_guard, 那就还没有进入 backfill, 只是在赌一次重跑。

4. Backfill plan 最少包含什么

```
{
  "partition_key": "2026-04-17",
  "window_start": "2026-04-17T00:00:00+00:00",
  "window_end": "2026-04-17T23:59:59.999999+00:00",
  "mode": "dry-run",
  "upstream_manifest_id": "week06-seed-manifest-001",
  "expected_input_count": 3,
  "current_output_count": 0,
  "gap_reason": "output_lag",
  "proposed_action": "dry_run_ticket_ingest_for_partition",
  "idempotency_guard": "ticket_id primary key plus raw source_fingerprint conflict guard",
  "downstream_impact": "Structured core only; Week04/Week05 remain observation-only in Week06",
  "operator": "student-devbox",
  "timestamp": "2026-04-17T10:00:00+00:00",
  "report_path": "reports/week06/backfill/backfill_plan_2026-04-17.json"
}
```

这段对齐当前项目里的 `pipelines/data_factory/backfill_plan.py`: 课程默认只做 dry-run planning, 不执行破坏性历史重写。字段名如果和项目代码发生差异, 以项目 runbook 和实现为准。

字段	保护什么	缺失会怎样
partition_key	补数边界	变成盲目重跑
window_start / window_end	时间范围	无法复核输入是否属于这个窗口
upstream_manifest_id	输入来源	无法 replay, 也无法解释读了哪版输入声明
expected_input_count	应读多少	无法判断缺口是否真实存在
current_output_count	现在产出多少	无法判断影响范围
gap_reason	为什么需要补	只能靠口头解释
proposed_action	打算怎么补	下游不知道你会不会写坏数据
idempotency_guard	防重复写入	可能双写、重复 ticket、污染 KPI
downstream_impact	下游是否阻断	误放行 Week07 / Week08 / BI 消费
operator / timestamp	谁在什么时候操作	事故复盘没有责任链
report_path	证据位置	无法交接给审阅者

5. 安全制造缺口

实验里不要危险删除主表。更稳的方式是让缺口停留在报告、manifest 或 dry-run 层面，而不是破坏长期环境。

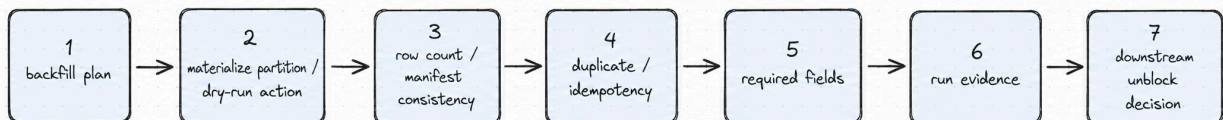
做法	是否推荐	原因
删除主表数据	不推荐	破坏长期环境，容易影响后续课程
删除某个 report 文件	推荐	可恢复、低风险，适合演示缺失证据
构造缺字段 manifest	推荐	适合演示 check 失败
使用 dry-run 只生成 plan	推荐	不写坏数据，适合课堂复盘
直接全量 rerun	不推荐	无法说明影响边界

项目侧 dry-run 命令已经落地：

```
# 可执行：已与项目 runbook 对齐
docker compose --profile tools --env-file infra/env/.env.local -f infra/docker-compose.yml run
--rm devbox \
    python -m pipelines.data_factory.backfill_plan --partition 2026-04-17 --mode dry-run
```

常见错误：如果输出提示 `Unsupported Week06 backfill mode`，检查 `--mode` 是否写成了 `dry-run`；如果 `expected input` 为 0，先确认 `seed` 数据中是否真的有 `2026-04-17` 的记录。

6. 补数后的验收顺序不能乱



这张图不是为了背流程，而是为了防止“补完就放行”。先 plan，再 run；先 row count，再 duplicate；先 required fields，再 run evidence；最后才是 downstream decision。没有 evidence，不允许口头 unblock。

! 核心判断

没有 checks 和 evidence 的 backfill，只是又跑了一遍；有边界、有验证、有证据的 backfill，才是工程恢复动作。

Week06 的验收顺序可以压缩成四句话：

1. plan 先证明“补什么”；
2. checks 证明“补对了吗”；
3. evidence 证明“谁、何时、用什么输入补的”；

4. downstream decision 决定“哪些下游可以继续消费”。

7. 企业里 backfill 最容易出事故的 6 个点

1. 没有 freeze 输入，补数时 source 还在变化；
2. 没有幂等键，同一个 ticket 被写两次；
3. 不写影响范围，下游不知道该 hold 哪些报表和索引；
4. 补完不跑 checks，只看脚本 exit code；
5. 下游没有 hold / unblock 决策，靠口头通知；
6. 没有保留补数证据，复盘时只能猜。

i Student Core 的最低标准

本周不要求做完整生产 backfill 平台，但必须能生成 dry-run backfill plan，并能解释每个字段保护的工程风险。

自检清单

- ☐ 我能解释为什么不默认全量重跑。
- ☐ 我能区分 retry / rerun / replay / restore / backfill。
- ☐ 我能解释 daily partition 为什么是 Student Core 默认。
- ☐ 我能写出一个 dry-run backfill plan，并说明每个字段保护什么。
- ☐ 我知道安全缺口模拟不能破坏主表。
- ☐ 我知道 backfill 后必须进入 checks 和 evidence。

课后最小行动

补一份恢复策略：

```
## Week06 recovery strategy
```

- 默认 partition:
- 允许 backfill 的资产:
- 禁止直接删除的对象:
- dry-run report path:
- backfill 后必须跑的 checks:
- downstream hold / unblock 规则:

延伸阅读

- Dagster: [Partitioning assets](#)¹
- Dagster: [Backfilling data](#)²
- Dagster: [Partitions and backfills](#)³
- 项目 runbook: [runbooks/week06-data-factory.md](#)

¹Dagster 的 partitioning 用于把资产划分成可独立管理、运行和补算的子集；课程默认 daily partition，是为了先让恢复边界足够清楚。

²Dagster backfill 面向缺失或需要更新的 partitions。Week06 课程默认只做 dry-run backfill planning，不执行破坏性历史重写。

³Partitions and backfills 是 Dagster 管理大数据集、增量处理和历史补算的核心能力；本课只使用其中最小可教学子集。